



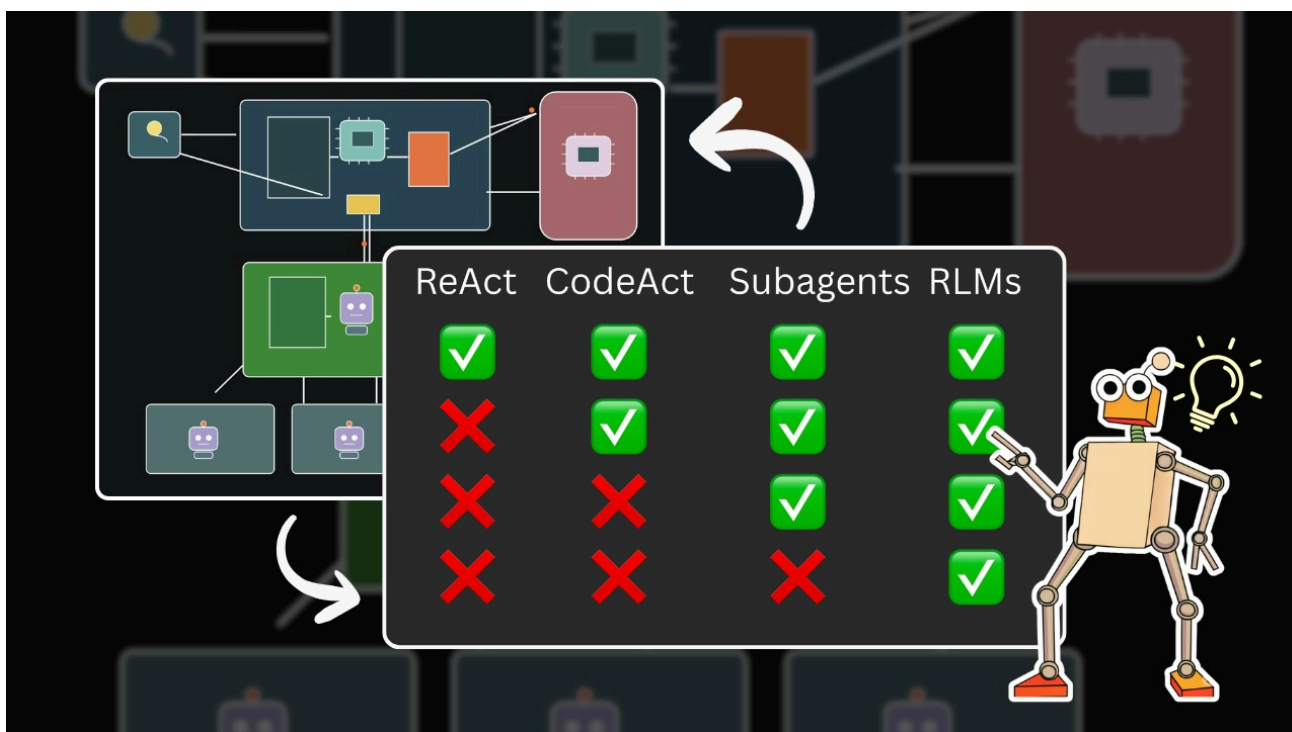
LARGE LANGUAGE MODELS

Recursive Language Models: An All-in-One Deep Dive

Exactly how does it differ from ReAct, CodeAct, Self-Loops, and Subagents?

Avishek Biswas

May 16, 2026 33 min read



In this article, you will learn what Recursive Language Models (RLMs) are, why they are winning all the long-context benchmarks right now, and understand how they are different from existing agentic harness designs!

And we are going to learn it by magnifying *one* simple case study.

I have spent a decent chunk of last month implementing RLMs, running benchmarks, and producing a 50-minute tutorial video on it. Throughout the process, I responded to 100+ questions on YouTube and X about RLMs. This article is a summary of what I learned answering those questions, and the specific nuances about RLMs that made me go “**a-ha!**”

Side note: Unless specified, all images used in this article were produced by the author. Free licensing.

RLMs are an *inference technique* where:

1. A language model interacts with arbitrarily long prompts through an external programmable environment or an *REPL*
2. The LLM can write code to *programmatically explore*, and create new *transformations* of the prompt
3. It can *recursively invoke sub-agents* to complete smaller subtasks – basically zoom in on specific strategic regions of the prompt and call separate LLMs to work on them
4. The subagent responses do not get automatically loaded into the parent agent's context, it *gets returned as symbols or variables inside the parent's REPL*
5. RLM agents can return responses in two ways – (a) *auto-regressively generated answers* like normal LLMs, and (b) *construct answers into a python variable* and return the variable instead.

The main reason Recursive Language Models feel inaccessible to a lot of the audience is that some of the ideas are actually quite counter-intuitive compared to existing methods (like ReAct, CodeAct, vanilla subagents, etc). The best way to understand RLMs is to first understand where those other methods fail, and realize the single missing piece in agentic harnesses.

The idea of passing context around by reference, instead of replicating it.

1. Of all the complicated experiments I ran...

... the most enlightening was this silly experiment where I asked an RLM to:

“Generate 50 names of fruits and count the number of R in each, return as a dictionary.”

And a more advanced variation (let’s call it Problem 2):

“Generate a dictionary of different categories: fruits, countries, animals. For each category, generate 50 names of that and count the number of R in each, return as a nested dictionary.”

For problem 1, the expected output is something like:

```
{"strawberry": 3, "berry": 2, ... "grape": 1}
```

And for problem 2, it is something like:

```
{  
  "fruits": {"strawberry": 3, "berry": 2, ... "grape": 1, ...},  
  "countries": {"united states of america": 1, "russia": 1, ...},  
  "animals": {"kangaroo": 1, "tiger": 1, ... "deer": 1, ...}  
}
```

I know it is a silly problem, but the way an RLM solves it is fundamentally different from other architectures like ReAct or CodeAct.

And understanding how each method solves this toy problem is all you are going to need to appreciate the beauty of RLMs.

Let's begin!

2. The Agentic Landscape

2.1 Direct Generation

The first method is just direct generation. The LLM “thinks” about the user’s request and auto-regressively generates a dictionary. No harness, no scaffold, just direct next-token prediction in a loop.

Problems with this approach:

- LLM has no way to verify if it is mathematically correct
- LLM will likely be wrong because, fundamentally, alphabet counting is not a “next word prediction” problem.
- Chances of hallucination or errors are extremely high, even if the underlying LLM is intelligent.

2.2 ReAct (Reasoning and Acting)

ReAct is a reasoning-and-acting loop where the LLM thinks about the problem first (chain-of-thought) and then generates a tool

call. Basically, in the system prompt, we pass a list of “function names”, and instructions about how to call them.

For example, you could give a simple tool to the LLM that is just:

```
def count_alphabets_in_word(word: string, alphabet: string) -> int
```

Using the above idea, the ReAct agent will be able to do the following:

- Generate a list of fruit names
- Use the tool to pass each fruit name and receive the output integer
- From its output memory, reconstruct the dictionary of which fruit got what count and then return.
- The stack trace of such a transaction would look like this:

User

Generate a dictionary with 50 fruits and the number of 'r' in each

Assistant

<think> 50 fruit names are: strawberry, berry, grape, ... </think>

Assistant

count_alphabets_in_word("strawberry", "r")

Tool_Out(executes our function)

3

Assistant

count_alphabets_in_word("berry", "r") ## Tool call executed!

Tool_Out(executes our function)

2

. . .

Assistant

```
<think> I now have everything I need in my message history,  
let's construct that dictionary </think>  
{ "strawberry": 3, "berry": 2, .... }
```

You see what the problems are, right? First, you have to define a function `count_alphabet_in_r` beforehand for this specific use-case. If you don't define a function, the agent just falls back to the old way (i.e. straight generation of alphabet counts)!

This guarantees that the LLM has some hint about what the output is, but the LLM still has to generate the tokens one at a time from its message history.

The LLM still has to remember the counts of each word, and reproduce it verbatim from memory. Transmission errors can still happen during this stage.

The problem compounds when you extend it to the multi-category setting of Problem 2. LLM has to repeat a long trace of function calls and remember what happened at each turn, and generate the answers token by token.

As an dev, **ReAct is great** if you developing narrow applications, where you want the agent to have access to specific tools (web search, document search,

calculator, terminal access, file edit, diff applier etc), but you will rarely develop a general agent and optimize for niche skills like these.

Basically, for general agents, only those universal tools are good. You won't write tools like `count_alphabet_in_r` unless you specifically know your users will want it.

What if the LLM could create its own tools?

2.3 CodeAct

CodeAct allows the LLM to write code and execute it.

Meaning you (the human) won't need to write exact tools anymore. You can just give the LLM the ability to write any python code and execute it in a sandboxed terminal environment, read the results and generate the output.

It will go something like this:

User

Generate a dictionary with 50 fruits and the number of 'r' in each

Assistant

<think> Okay let's write some python code for this. </think>

```
python -c '  
fruits = [ 'strawberry', 'berry' 'grape', .... ]  
count_r = { k: sum(1 for c in fruit if c == 'r') for k, f in fruit  
print("Number of fruits: ", len(fruits)) print("Counts: " , count_
```

Tool Output (Terminal Output)

Number of fruits: 50

Counts are: {"strawberry": 3, "berry": 2}

Assistant

<think> Okay, I have read the terminal output,
let me return write it down again to return the output </think>
{ "strawberry": 3, "berry": 2, }

So how CodeAct works is like:

- CodeAct reads the full user message (just like other methods we discussed before)
 - LLM thinks, writes, and runs code, or executes bash commands!
 - LLM loads the output of the code into its context window
 - Generate the result given what it read.
-

CodeAct is susceptible to the same transmission errors that we talked about in ReAct. Because the LLM still has to reproduce the answer verbatim from it's memory. The advantage of CodeAct (over ReAct) is that you (the human) do not need to preconfigure the available tools for the agent. The agent creates it's own tools (executable commands).

ME

Rule of thumbs for ReAct vs CodeAct:

- Use ReAct when you are working on narrow products and you know exactly which tools the AI needs to solve a problem.
- Use CodeAct when the domain is general.
- Remember, CodeAct will always run more slowly than ReAct because the LLM needs to spend time thinking and crafting its tools (whereas it gets the tools handed down by the user in ReAct).

Once again, the problem compounds when you extend it to the multi-category setting of Problem 2. The issue with Problem 2 is that the AI needs to keep track of too many internal states. It has to remember 150 different names across 3 different categories (fruits, countries, animals), and the number of 'r' in each word.

What if you could divide and conquer these three categories?

That is, have one agent work on fruits, one on countries, and one on animals?

2.4 CodeAct + Subagents

Now we are talking about some serious power!

- Subagent architectures are rather simple. There is a main agent, and they can launch smaller agents to perform sub-tasks.
- Each subagent is also a CodeAct agent that does whatever tasks they are assigned and returns output to the main agent.

- The **main agent loads these outputs straight into context** and performs the next unit of action

Understanding ALL of what I said above is integral to understanding the RLM architecture (coming up a bit later).

► More details about subagents and why they are useful

We already know the subagent architecture will easily solve Problem 1 with num_subagent = 0 (vanilla CodeAct), so let's actually see how it will work on Problem 2.

User

Generate a dictionary of different categories: fruits, countries,

Assistant

<think> Let's call some subagents and divide tasks among them </th

call_subagent("Return a dictionary of 50 fruit names and number of

Subagent (A new code-act module)

{"strawberry": 3, "berry": 2}

Assistant

call_subagent("Return a dictionary of 50 countries names and numbe

Subagent

{"france": 1, "russia": 1}

Assistant

call_subagent("Return a dictionary of 50 animals names and number

Subagent

{"kangaroo": 1, "deer": 1}

Assistant

<think> I have responses from all subagents, now I will write the
{

```
"fruits": { "strawberry": 3, "berry": 2, .... },  
"countries": { "france": 1, "russia": 1 .... }  
"animals": { "kangaroo": 1, "deer": 1 .... }  
}
```

We made a lot of cool progress. CodeAct + Subagent can write arbitrary code to arbitrary things, but it still must:

- READ the entire user prompt into its context window
- READ the entire subagent output into its context window
- Autoregressively WRITE the final output (after processing information returned by past tool calls and subagents)

The struggle is two-fold:

1. LLM needs to remember all the past tool call results
2. LLM needs to regurgitate the results in the correct format during output.

What if we allowed the LLM to write its results in an intermediate file, so it does not forget?

2.5 CodeAct + Subagents + File System

This is one of the most powerful architectures!

You give the LLM access to a special tool – `write_file` and `read_file`

You instruct the agent to write intermediate results to a persistent file system using these tools (or straight up using the `operator` inside a bash terminal). This helps the agent to

checkpoint progress so that it can load old states later whenever required!

Having file system access has a few caveats:

- More tool calls/read operations
 - Easier to remember things and not lose touch with reality
 - The transmission problem still exists: the LLM needs to read the file in the end and reproduce it verbatim (assuming that's a strict requirement)
-

What all of these solutions are missing is a simple feature:

Pass By Reference

It's an old programming concept where instead of passing a copy of variables back and forth between modules (or, in this case, agents) – you pass a reference to the variable.

That is what RLMS do.

3. Recursive Language Models

RLMs are a scaffold that calls LLMs a certain way to make them achieve tasks. Remember, a scaffold is an external system that prompts the LLMs in specific ways to make it do things, manage it's context, and step by step achieve a larger more complex task.

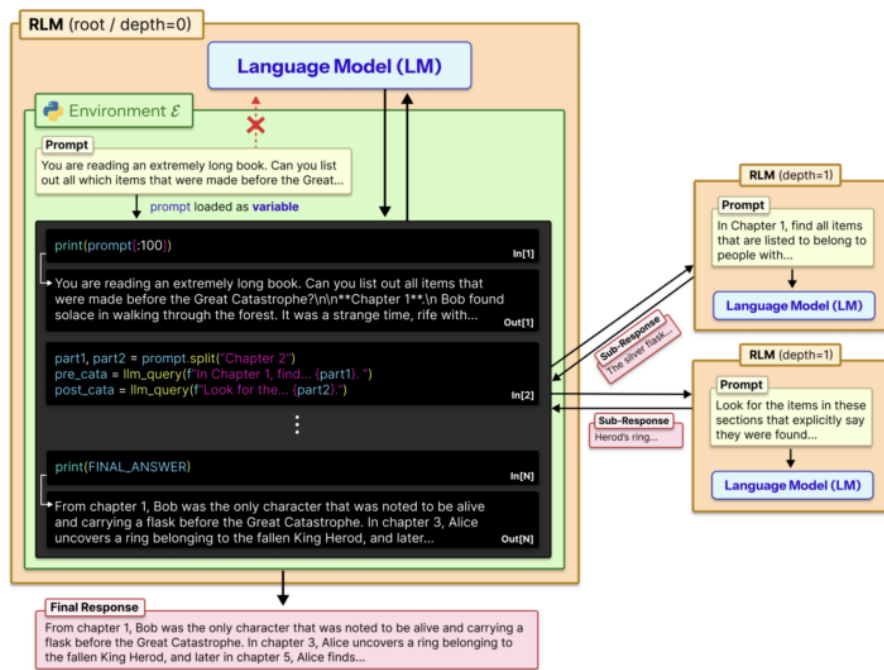


Figure 2: A Recursive Language Model (RLM) treats prompts as part of the environment. It loads the input prompt as a variable inside a REPL environment $\mathcal{E}$ and writes code to peek into, decompose, and invoke itself recursively over programmatic snippets of the variable.

From the RLM paper (<https://arxiv.org/abs/2512.24601>)

These are 4 points that explain what RLMs do:

- A language model interacts with arbitrarily long prompts through an external programmable environment or an **REPL**. Printed outputs are truncated at the scaffold layer.
- The LLM can write code to **programmatically explore** and create new **transformations** of the prompt
- It can **recursively invoke sub-agents** to complete smaller subtasks. The subagent responses do not get automatically loaded into the parent agent's context, it **gets returned as symbols or variables inside the parent's REPL**

- RLM agents can return responses in two ways: (a) auto-regressively generated answers like normal LLMs, and (b) construct answers into a Python ***variable*** and return the variable instead.

Let's break down each concept step by step.

3.1 The REPL

A REPL is a Read-Eval-Print-Loop. Think of it like a Jupyter notebook.

- You can have access to a *Python variable* called context where the user's query is kept.
- You can write commands to look at this context. For example, whenever the LLM issues a print statement, the live Python kernel prints out the expression.
- The LLM can iteratively read outputs to load new information into its context. Then decide on future action.
- REPL can also run in an isolated sandbox with configurable file-system permissions, so the LLM cannot impact the user's actual files. This is a security decision more than anything.

Here is an example of how an RLM run will “start”

- Before any LLM gets called, we will start a Python sandbox environment. *You can do this by running a pyodide instance inside Deno.js.*

- The Python runtime initializes with a special variable called “context” that contains the user’s prompt.
- What we pass into the LLM is NOT the content of the context, but just the fact that it has access to a REPL and there is a variable called context present in it. The LLM can run `print(context)` inside the REPL to view the prompt.

Here is an example trajectory:

System

You have access to a REPL python environment.
 Your task is stored in a variable called ``context``.
 You can issue print statements.
 Print displays truncated sections of the variable (upto 200 words)
 Find out what the task is about.
 Generate your code inside ````repl ... blocks`
 When ready to answer, submit your result using: `FINAL(answer)`

Assistant

`<think>` Let me print out the context to find out about my task `</t`

```
```repl
print(context)
```
```

REPL Output (executes Assistant code)

"Generate a dictionary containing 50 names of fruits
 and count the number of r in each"

The way the user prompt makes it into the LLM’s context window is not by how we pass it! The LLM makes a deliberate decision to read it from the environment.

In our case, the user's prompt is simple and short. But remember, the user's prompt can be arbitrarily long. For example, in one of my test cases, I input the complete transcripts of 300 Lex Fridman podcasts as a string that contained nearly 10M tokens.

The print statement in the REPL environment does not return the full output dump! Instead, it truncates the output to a fixed length and returns it.

Even if the RLM tries to overload itself with sensory information, we explicitly prevent the RLM from doing so by truncating the terminal output.

The LLM can always explore slices of the prompt deliberately too:

```
# Assistant
```repl
print(context[:200])
```

# REPL Output
** First 200 characters **

# Assistant
```repl
print(context[300:600])
```

# REPL Output
** 300 - 600th slice **
```

3.2 Programmatic Exploration

The LLM can also issue regex, find, and any other transformation code to extract information and store it in a variable. Remember, variables persist across execution calls because that is what an REPL does – it's a persistent Python runtime (imagine how Jupyter Notebook/IPykernel works)

```
x = re.match(...)\ny = context[30:90].split(",")\nprint(len(y))
```

- The LLM's prompt contains instructions to explore the prompt space and think about how it can wrangle the data to do its task.
- It is like how data scientists working on a fresh CSV dump of a housing prices dataset will print out random things into a Jupyter notebook to *understand* what they are dealing with.
- While exploring, the LLM can also create new variables inside the Python runtime that contain important transformations of the data!
- Remember, Python variables persist across different REPL execution calls. **I keep coming back to the Jupyter Notebook example because you must make this connection.** Each time the LLM writes a block of code and executes is equivalent to us humans writing a block of code and executing a cell!

Here is an example of RLM analyzing transcripts from Lex Fridman podcasts:

AVB 

@neural_avb · [Follow](#)



New RLM trajectory that blew my mind! I will use this one as the main example in the YT tutorial.

I passed in a CSV containing transcripts of 320 episodes of the Lex Fridman podcast and asked it to find what his first 10 ML guests had to say about AGI.

The context had [Show more](#)

Watch on X

8:22 AM · Feb 16, 2026



564



Reply



Copy link

[Read 34 replies](#)

Example explorations or transformations of context can be:

- LLM extracts an underlying CSV structure and puts the data into a pandas dataframe to process easier later
- The LLM extracts specific sections from a markdown file and creates a dictionary of subchapter_title -> subchapter texts
- The LLM issues regexes or find statements to search for keywords within the context (basic keyword search)

- The exploration stage is all about distilling the complete prompt into smaller, useful variables.

For our Problem 1, though, the task is straightforward, so the LLM's exploratory task is rather easy.

```
# Assistant
```

```
```repl
print(context)
```
```

```
# REPL Output
```

```
Generate a dictionary containing 50 names of fruits and
count the number of r in each
```

```
# Assistant
```

```
<think>okay I need to generate 50 fruit names</think>
```

```
```repl
FRUIT_NAMES = ['grape', 'strawberry', 'berry',]
print(len(FRUIT_NAMES)) # let's check if it is 50
```
```

```
# REPL Output
```

```
52
```

```
# Assistant
```

```
<think> Okay I added 2 extra by mistake.
Let me pick first 50 and check again.
</think>
```

```
```repl
FRUIT_NAMES_50 = FRUIT_NAMES[:50] # Just take first 50
assert len(FRUIT_NAMES_50) == 50
print("Successful: ", len(FRUIT_NAMES_50))
```
```

```
[[REPL]] Successful: 50
```

Assistant

<think> Cool, I'll just calculate the r counts now </think>

```
```repl
```

```
fruit_r_count = { fruit: sum(['r' in f for f in fruit.lower()])
for fruit in FRUIT_NAMES_50 }
```

```
FINAL(fruit_r_count)
```

```
```
```

As you can see:

- The LLM can auto-adjust depending on what the REPL output is. In its second try, it already has access to the original 52-count `FRUIT_NAMES` variable, so it was able to reuse that variable to slice it into a `FRUIT_NAMES_50` variable!
 - If the assert statement fails, the LLM will receive a REPL error and work to fix the code!
 - The LLM does not need to READ the dictionary `fruit_r_count` at all! It can just straight away pass it back to the user.
 - **The `FINAL(.)` just returns the result of an expression straight from the REPL back to the output of the scaffold!**
-

This is the first time that we have discussed a path where an agent is able to return an output to the user **without** (a) reading the whole dictionary into the context (b) generating the dictionary token-by-token ,c) not use file systems at all (*in theory CodeAct could*

have written the dictionary in a file system and asked the user to read from there)

For this reason, RLM outputs are not bound by the context length of the LLM. They can return arbitrarily long outputs, as large as the Python variable can hold.

3.3 Recursive Subagents

We have talked about some cool parts of RLMs already, but we haven't even gotten to the recursive parts.

In RLMs, the recursive-ness is similar to subagents, but there are fundamental differences in how information gets shared between subagents that are different in RLMs.

- RLMs have access to a special function inside their REPL called `llm_query`
- `llm_query` inputs a single string.
- `llm_query` invokes a brand new REPL environment, completely fresh, and sets context = whatever the parent LM had passed into `llm_query`
- This child RLM must solve the problem and send it back using `FINAL`
- The child RLM output is not loaded automatically into the parent RLM's context. Instead, it is just another expression inside the Python REPL!

To understand all this, let's take Problem 2 from above.

System

```
.... # everything as before, plus
You are given a new function called llm_query
llm_query invokes new subagents to perform tasks
Your subagents are quite strong and can handle long tasks well!
To call subagents just run llm_query(sub_context)
```

Assistant

```
```repl
print(context)
```
```

REPL Output

Generate a dictionary of different categories: fruits, countries,
For each category, generate 50 names. And count the number of R in
return as a nested dictionary

Assistant

```
<think> Cool I will just launch subagents to capture the
R counting for each category and do the job </think>
```

And then execute the code below:

```
FRUIT_DICT = llm_query("generate a dictionary of 50 fruits
                        and the number of times r occurred")
COUNTRY_DICT = llm_query("generate a dictionary of 50 countries
                        and the number of times r occurred")
ANIMAL_DICT = llm_query("generate a dictionary of 50 animals
                        and the number of times r occurred")

for dictionary in [FRUIT_DICT, COUNTRY_DICT, ANIMAL_DICT]:
    assert isinstance(dictionary, dict) and len(dictionary) == 5

answer = {
    "fruits": FRUIT_DICT,
    "countries": COUNTRY_DICT,
    "animals": ANIMAL_DICT
}
```

FINAL(answer)

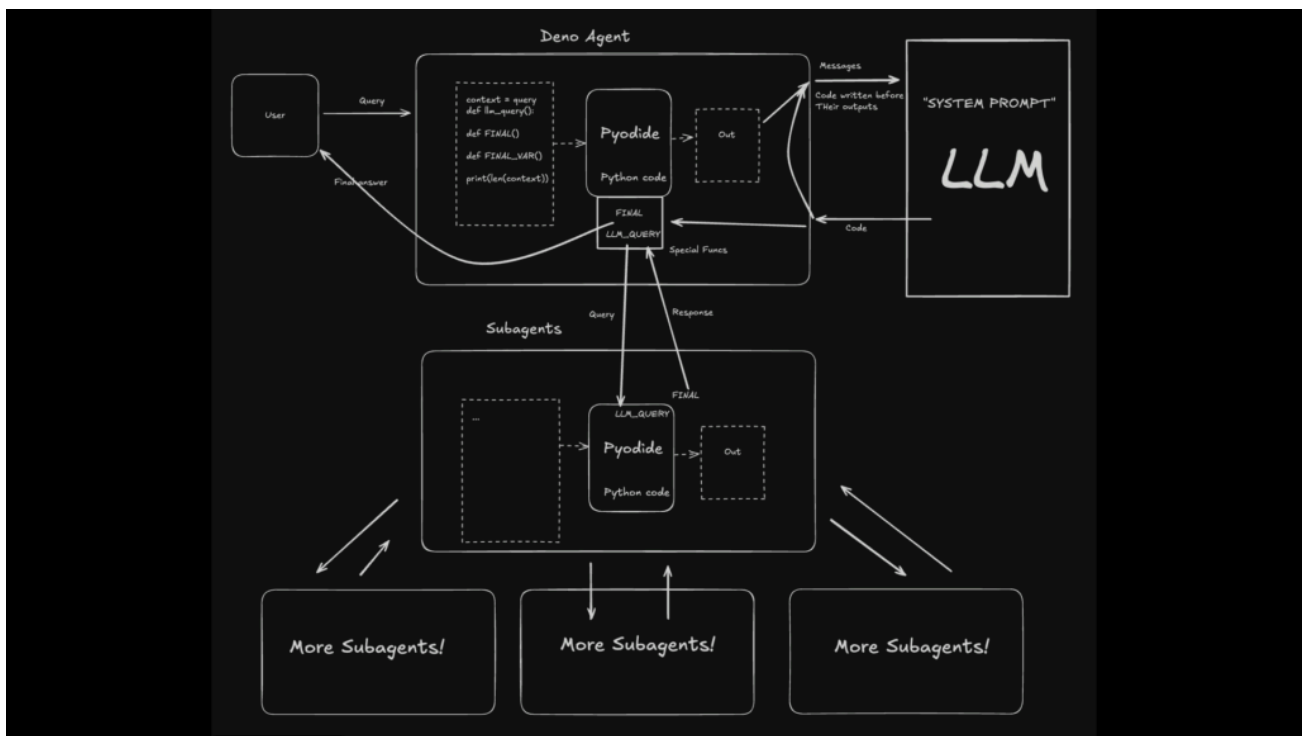
In practical RLM implementations, we will be able to parallelize these calls. Multiple subagents working in parallel on orthogonal tasks is not just super cool, but it actually gets a ton of stuff done really fast.

Notice what just happened.

- The LLM assigned 3 subagents the task of managing fruits, countries, and animals
- The subagents (as we saw previously) will return the answers calling `FINAL` in their own local REPL
- That outputs lands directly inside the `FRUIT_DICT`, `ANIMAL_DICT` and `COUNTRY_DICT` dictionaries of the main agent's REPL
- **The subagent outputs are entered into the REPL, they are not loaded directly into the context of the LLM (like how CodeAct or ReAct subagents worked).** To view the subagent outputs, the main agent needs to inspect it deliberately with `print` statements.

The main agent did not even need to:

- Load the entire subagent output into context
- Read any of the fruit names
- Generate the final output token by token from memory
- It **composed** an answer by forming the key symbols through recursive calls and delivering the final output as a composition.



The Basic RLM architecture with Deno and Pyodide

3.4 The RLM's Output Space

- RLMs can choose two ways to return their FINAL output.
- One, it can compose answers into Python variables and return them (like the example above)
- Or it can generate a response on its own autoregressively, just like a normal LLM

In the case below, the output was autoregressively generated.

```
# Assistant
print(context)
```

```
# REPL Output
Capital of France?
```

```
# Assistant
FINAL('Capital of France is Paris')
```

In the case above, the output was computed in Python, and the contents of that variable were returned.

```
# Assistant
```

```
print(context)
```

```
# REPL Output
```

```
Today's date?
```

```
# Assistant
```

```
```repl
```

```
from datetime import date
```

```
today = date.today()
```

```
FINAL(today)
```

```
```
```

These two modes of generating answers open a huge opportunity for RLMs:

1. They can programmatically explore using regexes, find operations using regular Python
2. They can create small variables to save work (they are inside a REPL, so old work is never lost)
3. They can recursively call agents to summarize.
4. Subagents can be parallel or sequential. The LLM intelligently decides this. A reason the RLM may want to call subagents sequentially is if it needs to do a running summary of a long context text that needs prior information.
5. They can also use external tools, but you have to expose them through your sandbox layer (Deno, for example)

To understand how RLMs work in more visual detail, how they can be implemented from scratch, and see some real trajectories

where it attacks real world problems, check out this video tutorial:

Recursive Language Models (RLMs) - Let's build the coolest

Neural Breakdown with AVB



Watch on

Check out my open-source implementation of RLMs; it comes with a TUI log viewer for recursive traces.

<https://github.com/avbiswas/fast-rlm>

Here is the full system prompt that I used for my RLM implementation. This will reveal a lot!

► **Click here to reveal the full System Prompt (it's hidden because it's long)**. You can find the author-recommended prompt in the RLM paper (linked below). The prompt here was repurposed from the paper's prompt, with a few additional few-shot examples and instructions that reduced failure states on open-source models (tested on Minimax-M2.7, GLM-5.1)

ou can study the full paper here: <https://arxiv.org/abs/2512.24601>

Or with an AI: <https://paperbreakdown.com/abs/2512.24601>

4. Why does this work so well?

1. **Focused attention:** Instead of attending to all token pairs in a huge input, RLM allows the model to target specific sections to load into the context. **RLM loads context BY choice, not forcefully like ReAct or CodeAct does. By combining information from multiple different sections of the prompt.**
2. **Multi-step reasoning:** Many tasks are naturally recursive (multi-hop QA, codebase search, multi-document summarization). RLMs natively match the multi-task structure **RLMs can just iteratively refine its plan through simply printing various slices of the context and loading it into the context.**
3. **Robustness to noise:** When 99% of the input is irrelevant, recursive search avoids “attention dilution.” A smart model will intelligently load parts of the prompts that is likely to give it the most information. **Cherry picking what context to load into memory is a sign of intelligence!**
4. **Results are composable variables:** Sub-agent answers are not loaded directly into the LLM’s context; they are returned as symbols inside the Python REPL, and the agent can choose to either peek into the results or directly use them. **They can compose results directly out of subagent responses without fully reading them**
5. **Arbitrarily long outputs:** Remember, RLMs do not need to auto-regressively generate answers; they can instead construct answers inside a Python variable – this means the

model can, in theory, return infinitely long outputs.

Summarization tasks are still autoregressive for the most part.

6. **Cost savings:** Because the model decides what to read and when to recurse, you often pay for what you need, not for scanning everything. The RLM paper shows the results of how cheap these experiments can be to run compared to other methods. Low cost on prompt input tokens! And depending on the problem, low cost on completion tokens.
7. **Subagents still hit KV Caches:** Subagents perform tasks one step at a time, so their system prompt and past messages do not change. You are hitting KV Caches 90% of the time, so your cost is low. **Subagents follow a simple user->assistant->user->assistant message template. Immediate KV cache benefits!**
8. **Separation of concerns :** Root LM acts as a “planner/orchestrator” while subagent LMs are “executors/workers” that do low-level work. You can pick different models to do these different tasks as well! **You can customize which model does what. Infact you can extend RLMs to pick what type of model should work on a subproblem**

Good coding models are naturally good at being RLM drivers. People are already training models on RLM harnesses, so I imagine this will only get better!

Thanks for reading!